NSI S6 : Les tableaux

Dans les séquences précédentes, vous avez déjà utilisés des tableaux. Le but de cette séquence est de :

- Lire et modifier les éléments d'un tableau grâce à leurs index ;
- Construire un tableau par compréhension ;
- Utiliser des tableaux pour représenter des matrices.

1. Lire et modifier les éléments d'un tableau grâce à leurs index :

Nous allons employer le terme "tableau". Pour parler de ces "tableaux" les concepteurs de Python ont choisi d'utiliser le terme de "list" ("liste" en français). Les tableaux sont, comme les tuples, des séquences, mais à la différence des tuples, ils sont modifiables (on parle d'objets "mutables").
On rappelle qu'une liste se déclare : L=[…,…,…,…]
On rappelle aussi que pour avoir le nombre d'éléments d'une liste, il faut faire len(L)

I) Écrire un programme qui créé une liste d'au moins 5 entiers [8, 5, 6, 1, 17] puis :
- afficher la valeur de L[4]
- modifier la liste en remplaçant L[1] par 17 et L[3] par la somme des cases voisines L[2] et L[4]
- afficher 5 fois la valeur du dernier terme de la liste

Sauver sous exo1.py puis tester.

II) Écrire un programme qui échange les valeurs de la première et de la dernière case d'une liste quelconque non vide et affiche le résultat.
On donne l'algorithme suivant :
#utilisation d'une variable auxiliaire
#aux=x
#x=y
#y=aux
Sauver sous exo2.py puis tester avec L=[6,12,18,5].

III) Écrire un programme qui crée une liste d'entiers, puis qui affiche la liste en colonne.
Sauver sous exo3.py puis tester avec L=[8,24,48,2,16].

IV) Écrire un programme qui crée une liste d'entiers, puis qui compte le nombre de multiples de 3 présents dans la liste.
Sauver sous exo4.py puis tester avec L=[8,24,48,2,16].

V) Écrire un programme qui crée une liste d'entiers, puis qui calcule la somme de toutes les valeurs paires de la liste.
Sauver sous exo5.py puis tester avec L=[8,24,48,2,16].

VI) Écrire un programme qui crée une liste d'entiers, puis qui calcule le maximum et le minimum des éléments de la liste. On prendra au départ min=L[0] et max=L[0] puis on testera L[i]par rapport à min et max.
Sauver sous exo6.py puis tester avec L=[8,24,48,2,16].

VII) Écrire un programme qui crée une liste d'entiers, puis créer un booléen vraie si la moyenne arithmétique des valeurs de la liste est supérieure ou égale à 10.
Sauver sous exo7.py puis tester avec L=[8,24,48,2,16].

VIII) Écrire un programme qui crée une liste d'entiers, puis qui affiche la liste à l'envers (sans créer une nouvelle liste).
Sauver sous exo8.py puis tester avec L=[8,24,48,2,16].

IX) Écrire un programme qui crée une liste d'entiers, puis qui modifie la liste en augmentant de 1 la valeur de chaque élément de la liste.
Sauver sous exo9.py puis tester avec L=[13,27,50,42,3].

X) Écrire un programme qui crée une liste d'entiers, puis qui modifie la liste par permutation circulaire (Le premier élément se retrouve en dernière position, le deuxième en première etc….)
Sauver sous exo10.py puis tester avec L=[8,24,42,16].

XI) Écrire un programme qui crée une liste d'entiers, puis qui modifie la liste par miroir.
Sauver sous exo11.py puis tester avec L=[8, 24, 42, 16] puis avec L = [8, 24, 48, 2, 16, 12].

XII) On dispose d'une liste L non vide de nombres. Créer une liste LC contenant les carrés des nombres de L.Pour cela on utilisera la méthode LC.append(…….)
Sauver sous exo12.py puis tester avec L=[8,27,42,17,4].

XIII) On dispose maintenant de deux listes L1 et L2. Il s'agit de créer une liste :
- L3 identiques à L1 dans laquelle on a supprimé la première valeur
- L4 contenant les valeurs L1 puis celles de L2

Sauver sous exo13.py puis tester avec L1 = [8.3, 27.2, 42, 17, 4.1] et L2 = [15, 5.9, 59, 10.1, 2]

2. Construire un tableau par compréhension :

Nous avons vu qu'il était possible de "remplir" un tableau en renseignant les éléments du tableau les uns après les autres ou encore à l'aide de la méthode "append".
Il est aussi possible d'obtenir exactement le même résultat en une seule ligne grâce à la compréhension de tableau.
Exemple : la création d'un tableau de 5 éléments dont les valeurs sont [0, 1, 2, 3, 4]
Par compréhension, on pourra écrire : mon_tab = [p for p in range(0, 5)]
Les compréhensions de tableau permettent de rajouter une condition (if) :
L = [1, 7, 9, 15, 5, 20, 10, 8]
mon_tab = [p for p in L if p > 10]
mon_tab ne gardera que les valeurs de L supérieur à 10 : [15, 20].

XIV) Écrire un programme qui crée une liste d'entiers (mon_tab) par compréhension qui valent le carré des éléments < 10.
Sauver sous exo14.py puis tester avec L=[1, 7, 9, 15, 5, 20, 10, 8].
Rappel : p**2 permet d'obtenir la valeur de p élevée au carrée.

XV) Écrire un programme qui crée une liste d'entiers (L) par compréhension de 0 à 15 puis une liste L_pair qui par compréhension calcule 2*n des valeurs n paire de L. Afficher L et L_pair
Sauver sous exo15.py puis tester.
Vous devriez obtenir :
L= [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15]
L_pair=[0, 4, 8, 12, 16, 20, 24, 28]

XVI) Écrire un programme qui crée une liste d'entiers (L) par compréhension de 0 à 15 puis les chiffres pairs seront multipliés par deux, les chiffres impairs par trois.
Sauver sous exo16.py puis tester.
Vous devriez obtenir :
L= [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15]
L_finale=[0, 3, 4, 9, 8, 15, 12, 21, 16, 27, 20, 33, 24, 39, 28, 45]

XVII) Écrire un programme qui crée une liste d'entiers L= [[0, 'a'], [2, 'b'], [3, 'c']] puis multiplier les nombres entiers par deux et les strings par trois.
Sauver sous exo17.py puis tester.
Vous devriez obtenir :
L= [[0, 'a'], [2, 'b'], [3, 'c']]
L_finale= [0, 'aaa', 4, 'bbb', 6, 'ccc']

3. Utiliser des tableaux pour représenter des matrices :

Chaque élément d'un tableau peut être un tableau, on parle de tableau de tableau.
Exemple : m = [[1, 3, 4], [5 ,6 ,8], [2, 1, 3], [7, 8, 15]]
Pour que cela soit plus lisible, on la code plutôt de la façon suivante :

```
m = [[1, 3, 4],
     [5 ,6 ,8],
     [2, 1, 3],
     [7, 8, 15]]
```

Il est évidemment possible d'utiliser les indices de position avec ces "tableaux de tableaux". Il faut considérer notre tableau de tableaux comme une matrice, c'est à dire en utilisant les notions de "ligne" et de "colonne". Dans la matrice ci-dessus :
1, 3, 4 constituent la première ligne
5, 6, 8 constituent la deuxième ligne
2, 1, 3 constituent la troisième ligne
7, 8, 15 constituent la quatrième ligne

1, 5, 2, 7 constituent la première colonne
3, 6, 1, 8 constituent la deuxième colonne
4, 8, 3, 15 constituent la troisième colonne
Pour cibler un élément particulier de la matrice, on utilise la notation avec "doubles crochets" : m[ligne][colonne] (sans perdre de vu que la première ligne et la première colonne ont pour indice 0).

Pour effectuer la somme de deux matrices :

$$\begin{pmatrix} a_{1,1} & \cdots & & \cdots & a_{1,n} \\ \vdots & & \vdots & & \vdots \\ a_{i,1} & \cdots & a_{i,j} & \cdots & a_{i,n} \\ \vdots & & \vdots & & \vdots \\ a_{m,1} & \cdots & & \cdots & a_{m,n} \end{pmatrix} + \begin{pmatrix} b_{1,1} & \cdots & b_{1,k} & \cdots & b_{1,n} \\ \vdots & & \vdots & & \vdots \\ & \cdots & b_{j,k} & \cdots & \\ \vdots & & \vdots & & \vdots \\ b_{n,1} & \cdots & b_{n,k} & \cdots & b_{n,p} \end{pmatrix} = \begin{pmatrix} c_{1,1} & & \vdots & & c_{1,p} \\ & & \vdots & & \\ \cdots & \cdots & c_{i,k} & & \\ & & & & \\ c_{m,1} & & & & c_{m,p} \end{pmatrix}$$

$$\text{Avec} \qquad C_{i,j} = A_{i,j} + B_{i,j}$$

XVIII) Ecrire un programme qui retourne la somme de deux matrice M1 et M2 de même dimension. Sauver sous exo18.py puis tester et tester avec M1= [[2 , 5 , 1] , [0 , 1 , 2]] et
M2 = [[20 , -5 , 11] , [10 , -1 , 5]].
Vous devriez obtenir :
M1=[[2, 5, 1], [0, 1, 2]]
M2=[[20, -5, 11], [10, -1, 5]]
M=[[22, 0, 12], [10, 0, 7]]